1. Modelo de datos implementado(models.py).

```python
from django.db import models

# Create your models here.

class Rol(models.Model):
    nombre = models.CharField(max_length=50, unique=True)
    descripcion = models.CharField(max_length=100)

    class Meta:
        db_table_comment = "Roles de usuario en la plataforma (cliente,
Administrador, Garzón)"

    def __str__(self):
        return self.nombre


class Usuario(models.Model):
    nombre = models.CharField(max_length=50)
    apellido = models.CharField(max_length=50)
    email = models.EmailField(max_length=150, unique=True)
    telefono = models.CharField(max_length=20)
    password_hash = models.CharField(max_length=255)
    fecha_registro = models.DateTimeField(auto_now_add=True)
    estado = models.BooleanField(default=False)
    rol = models.ForeignKey(Rol, on_delete=models.CASCADE)

    class Meta:
        db_table_comment = "información de los clientes y personal del
restaurante"

    def __str__(self):
        return f"{self.nombre} {self.apellido}"

class Restaurante(models.Model):
    nombre = models.CharField(max_length=100)
    direccion = models.CharField(max_length=100)
    telefono = models.CharField(max_length=15)
    capacidad_total = models.IntegerField()
    estado = models.BooleanField(default=True)

    class Meta:
        db_table_comment = "Datos principales de cada local o sucursal"
```

```
    def __str__(self):
        return f"{self.nombre}, ubicado en {self.direccion}"

class Sector(models.Model):
    nombre = models.CharField(max_length=100)
    descripcion = models.CharField(max_length=255, blank = True, null=True)
    restaurante = models.ForeignKey(Restaurante, on_delete=models.CASCADE)

    class Meta:
        db_table_comment = "ambientes disponibles dentro de cada restaurante
(Terraza, inferior, VIP, etc.)"

    def __str__(self):
        return f"{self.nombre} {self.restaurante.nombre}"

class Mesa(models.Model):
    codigo_mesa = models.CharField(max_length=10)
    capacidad_min = models.IntegerField()
    capacidad_max = models.IntegerField()
    estado = models.BooleanField(default=True)
    sector = models.ForeignKey(Sector, on_delete = models.CASCADE)

    class Meta:
        db_table_comment = "Mobiliario físico y capacidad asignable para cada
sector"

    def __str__(self):
        return f"{self.codigo_mesa} {self.sector.nombre}"

class Bloqueo(models.Model):
    fecha_inicio = models.DateTimeField()
    fecha_fin = models.DateTimeField()
    motivo = models.CharField(max_length=100)
    mesa = models.ForeignKey(Mesa, on_delete=models.CASCADE)

    class Meta:
        db_table_comment = "Excepciones de disponibilidad (puede ser por
mantención, avería o eventos privados)"

    def __str__(self):
        return f"Bloqueo: {self.mesa.codigo_mesa}: {self.motivo}"

class Reserva(models.Model):
    codigo_reserva = models.CharField(max_length=15, unique=True)
    fecha_reserva = models.DateField()
```

```
    hora_inicio = models.TimeField()
    hora_fin_estimada= models.TimeField()
    cantidad_personas = models.IntegerField()
    estado = models.CharField(max_length=20, default="PENDIENTE")
    comentario = models.CharField(max_length=255)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    usuario = models.ForeignKey(Usuario, on_delete=models.CASCADE)
    restaurante = models.ForeignKey(Restaurante, on_delete = models.CASCADE)

    mesas = models.ManyToManyField(Mesa, through="ReservaMesa",
related_name="reservas")
    class Meta:
        db_table_comment = "Registro central de las reservas realizadas por los
clientes"

    def __str__(self):
        return self.codigo_reserva

class CambioReserva(models.Model):
    estado_anterior = models.CharField(max_length=20)
    estado_actual = models.CharField(max_length=20)
    motivo_cambio = models.CharField(max_length=255)
    fecha_cambio = models.DateTimeField(auto_now_add=True)
    reserva = models.ForeignKey(Reserva, on_delete=models.CASCADE)
    usuario = models.ForeignKey(Usuario, on_delete = models.CASCADE)

    class Meta:
        db_table_comment = "Historial de auditoría para cambios de estado en las
reservas"

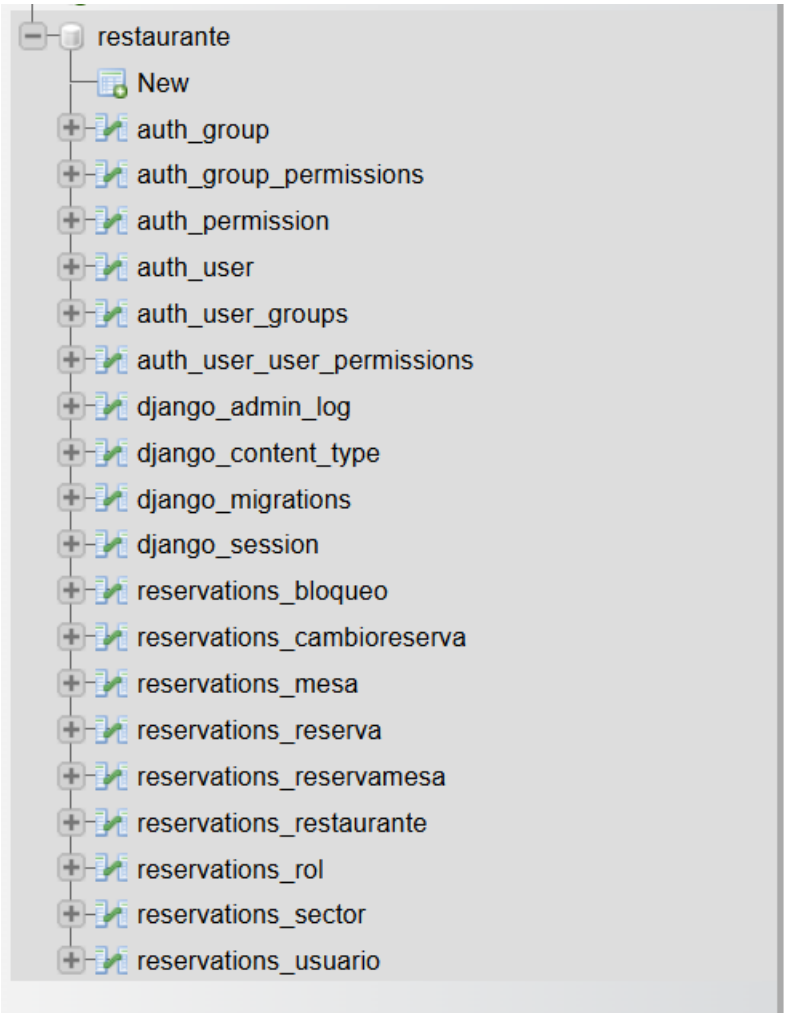
    def __str__(self):
        return f"cambio en {self.reserva.codigo_reserva}, cambio de estado:
{self.estado_anterior} -> {self.estado_actual} por el motivo de
{self.motivo_cambio}"

class ReservaMesa(models.Model):
    reserva = models.ForeignKey(Reserva, on_delete = models.CASCADE)
    mesa = models.ForeignKey(Mesa, on_delete = models.CASCADE)

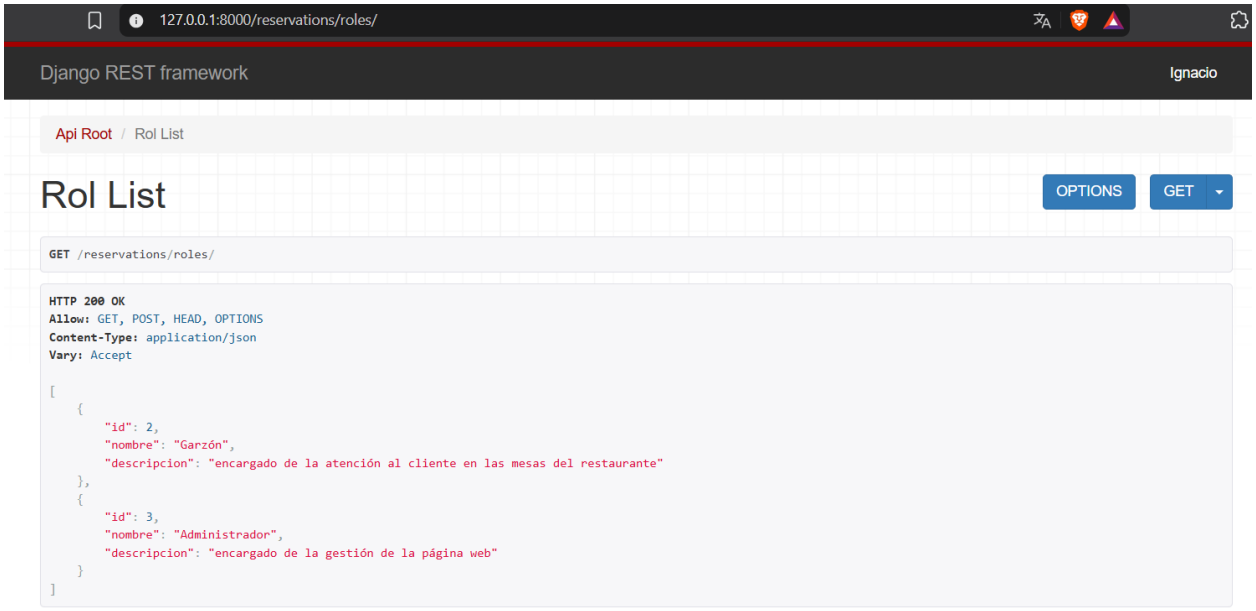
    class Meta:
        db_table_comment = "tabla intermedia para asociar una o múltiples mesas
físicas a una reserva"
        constraints = [
            models.UniqueConstraint(
                fields=["reserva", "mesa"],
```

```
                name = "unique_reserva_mesa"
            )
        ]
    def __str__(self):
        return f"Reserva {self.reserva.codigo_reserva} asignada a Mesa
{self.mesa.codigo_mesa}"
```

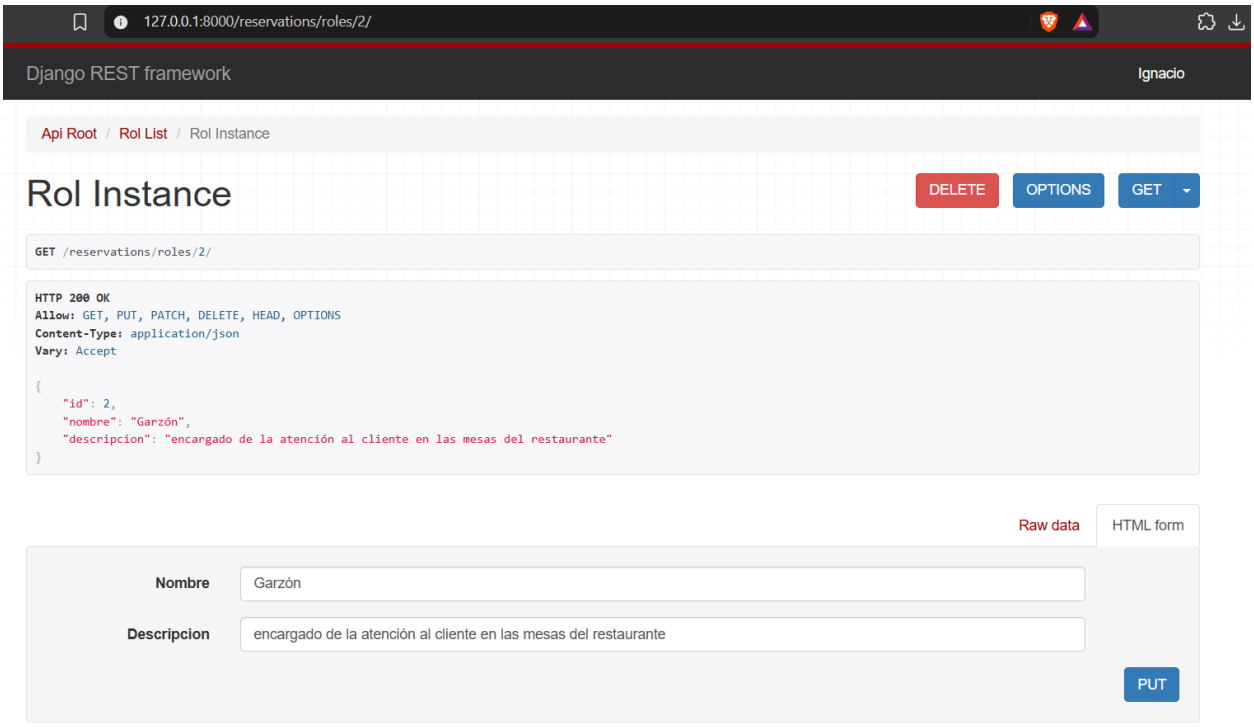
2. Migraciones generadas.



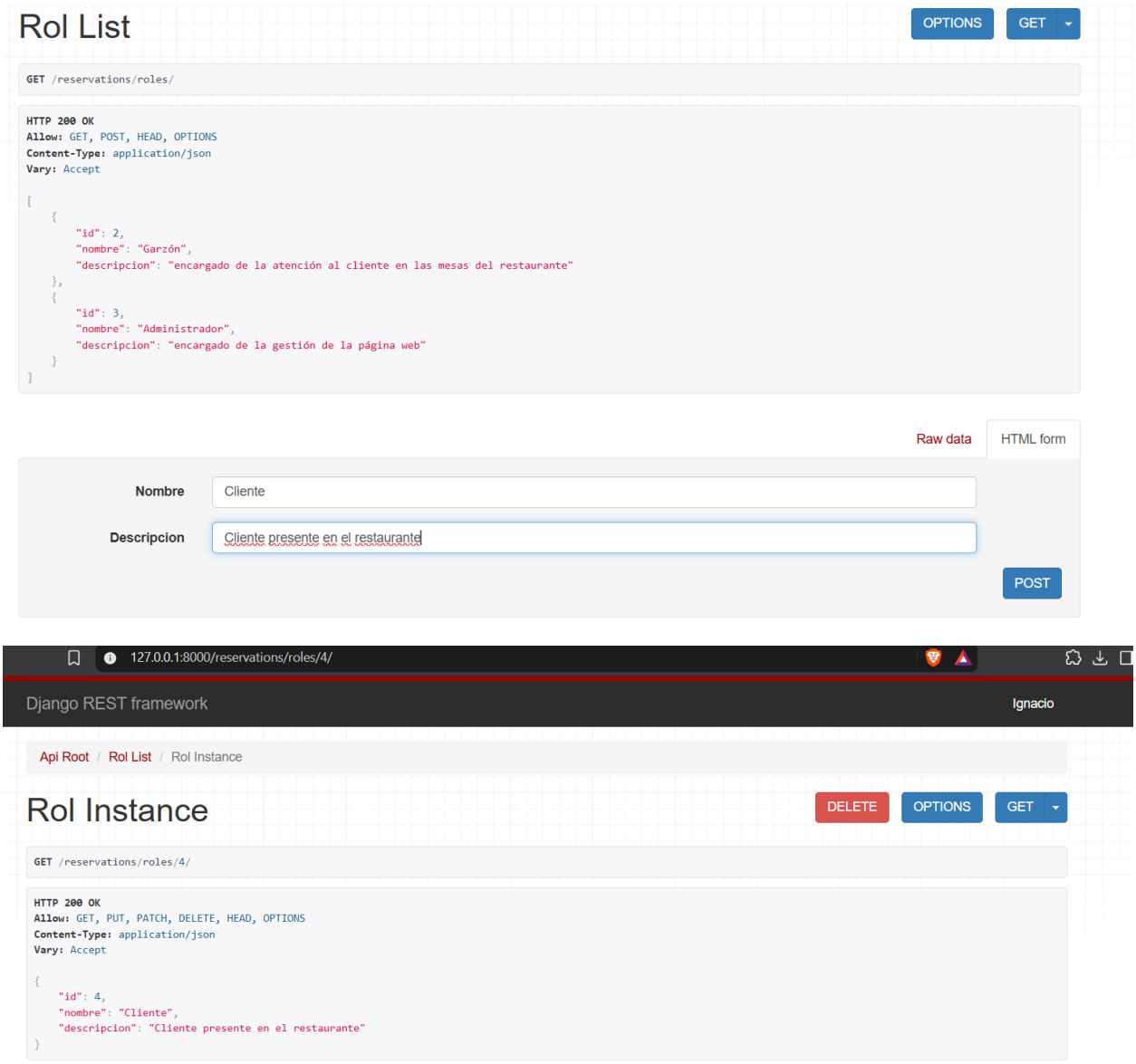
3. Consulta de registros mediante GET.



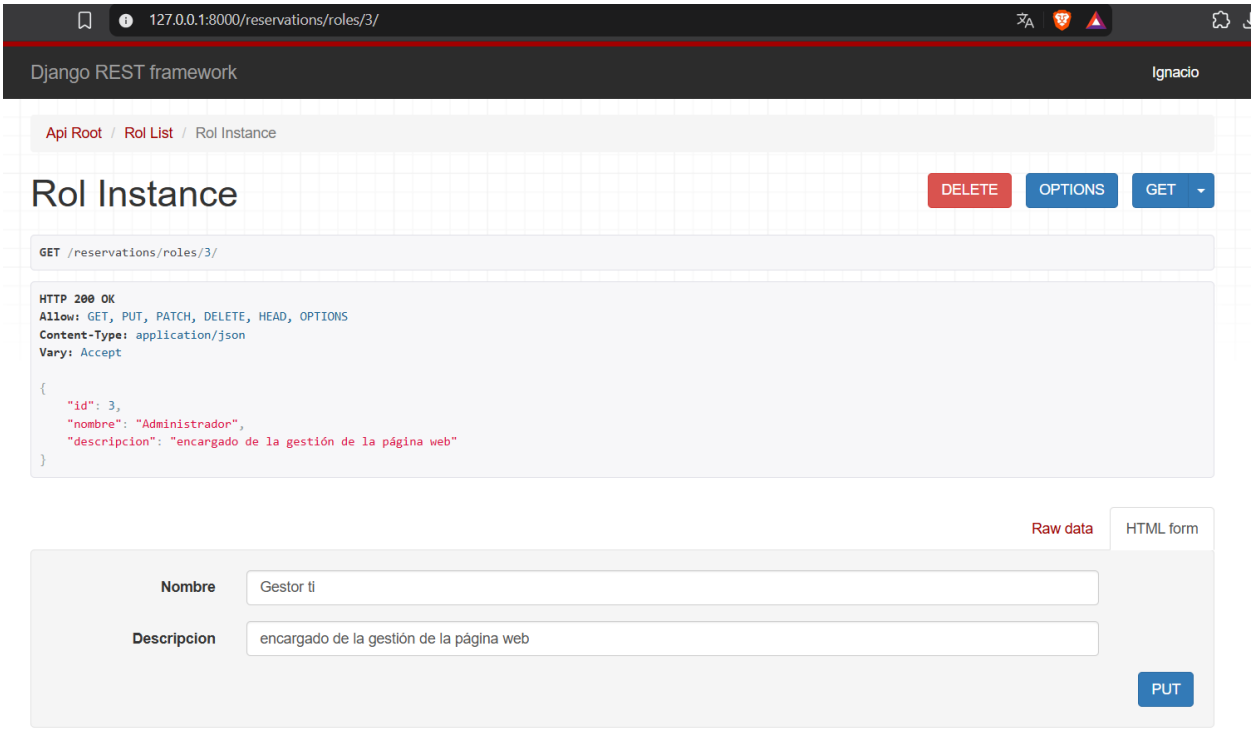
4. Consulta de detalle mediante GET por identificador.

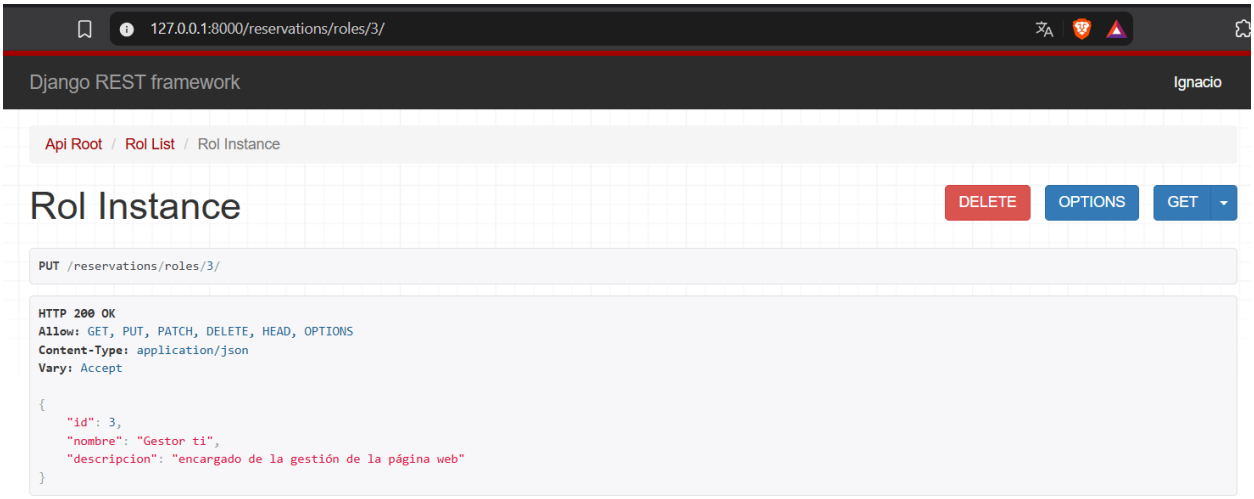


5. Creación de registros mediante POST.

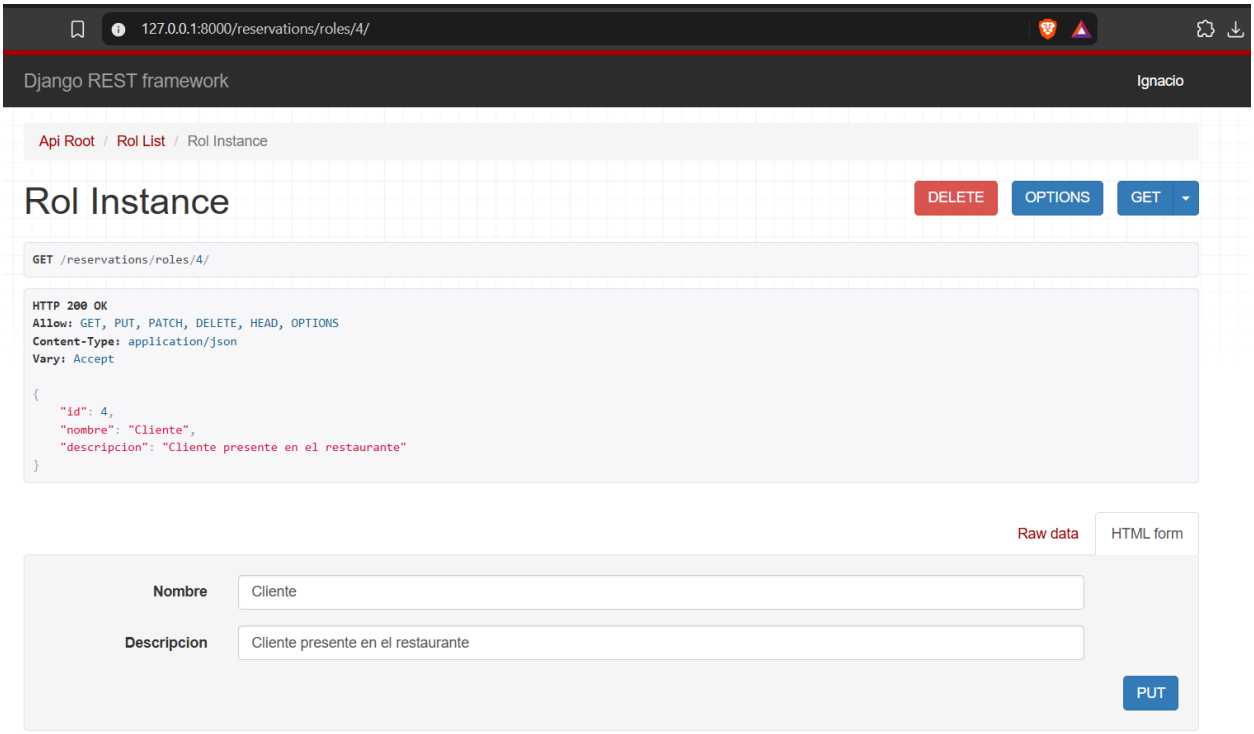


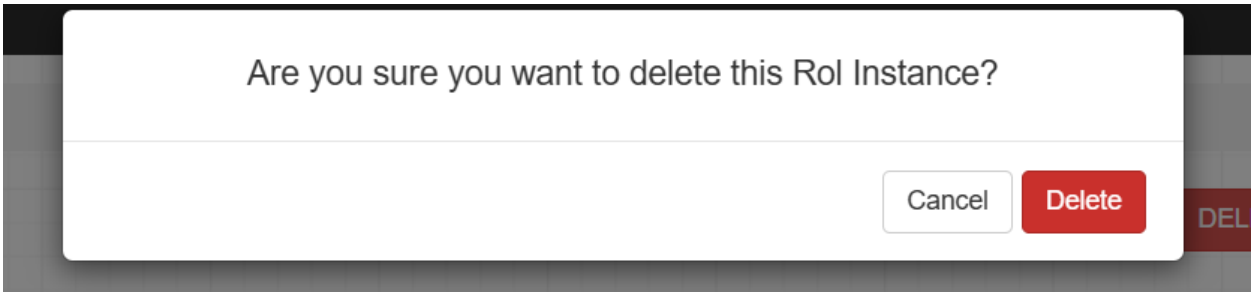
6. Modificación de registros mediante PUT o PATCH.

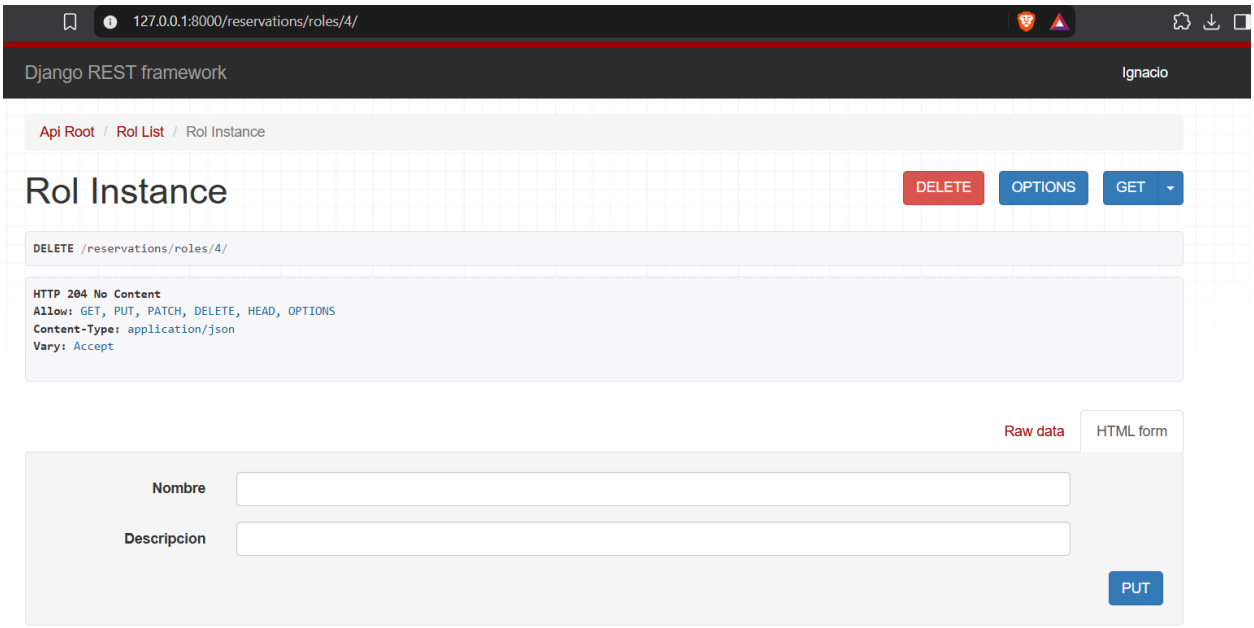




7. Eliminación de registros mediante DELETE.


127.0.0.1:8000/reservations/roles/4/
Django REST framework
Ignacio
Api Root / Rol List / Rol Instance
Rol Instance
DELETE
OPTIONS
GET
GET /reservations/roles/4/
HTTP 200 OK
Allow: GET, PUT, PATCH, DELETE, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept
{
"id": 4,
"nombre": "Cliente",
"descripcion": "Cliente presente en el restaurante"
}
Raw data
HTML form
Nombre
Cliente
Descripcion
Cliente presente en el restaurante
PUT


Are you sure you want to delete this Rol Instance?
Cancel
Delete



8. Uso de variables de entorno.

.env

```
DB_ENGINE=django.db.backends.mysql
DB_NAME=restaurante
DB_USER=Usuario
DB_PASSWORD=a.1234567
DB_HOST=127.0.0.1
DB_PORT=3306

SECRET_KEY='django-insecure-_r2t$(^b_3gr3(0u_h+$q0=o$&@pjpwbjede2-6pq7&tu9juul'

```

config > settings.py > ...

```
# See https://docs.djangoproject.com/en/6.1/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = os.getenv('SECRET_KEY')

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True
```

```
DATABASES = {
    'default': {
        'ENGINE': os.getenv('DB_ENGINE', 'django.db.backends.mysql'),
        'NAME': os.getenv('DB_NAME'),
        'USER': os.getenv('DB_USER'),
        'PASSWORD': os.getenv('DB_PASSWORD'),
        'HOST': os.getenv('DB_HOST', '127.0.0.1'),
        'PORT': os.getenv('DB_PORT', '3306')
    }
}
```

9. Repositorio GitHub actualizado.

https://github.com/Ignacioea/Gestor-de-Reservas-de-Restaurantes.git